Control Tower Workflow Guide

Part 1 of 7: What I inspected, the big picture and the system map

Based on your seven uploaded n8n exports: CT-01 to CT-07. Everything comes from the JSON. Sample data is marked **illustrative**. Where something cannot be known without running the workflow, it says so.

In this part	Sections
What I inspected (all seven files, node counts, triggers)	1
Four findings worth knowing straight away	2
The real-life analogy: a company operations control room	3
What the complete system does	4
Overall diagrams (main journey, support workflows)	5
How the workflows talk to each other	6
Roadmap for Parts 2-7	7

1. What I inspected

All seven files were opened and read. Five were already visible to me; CT-01 and CT-05 were read directly from the uploaded files. Counts below separate **working nodes** from **sticky notes** (yellow note boxes with no function).

Workflow	What starts it	Nodes: total / working / notes	Role in one line
CT-01 Main - Event Intake & Orchestration	Webhook POST `/ops-events`	59 / 52 / 7	The front door and main pipeline
CT-02 Sub - State & Audit Writer	Called by another workflow	12 / 9 / 3	Only place that changes an incident's status and writes the history log
CT-03 Sub - Dispatch Action	Called by another workflow	7 / 5 / 2	Posts the alert to Slack
CT-04 Global Error Handler	n8n Error Trigger	11 / 8 / 3	Catches workflows that crash
CT-05 Watchdog	Schedule	28 / 24 / 4	Retries failures, finds stuck incidents, spots outbreaks
CT-06 Human Recovery Endpoint	Webhook POST `/ops-recover`	10 / 7 / 3	Lets a person restart an escalated or failed incident
CT-07 Sub - HubSpot Outcome Sync	Called by another workflow	10 / 9 / 1	Writes the outcome as a note in HubSpot

- **Totals:** 7 workflows, 139 nodes (123 working). All seven are marked active.
- **Schedule detail:** CT-05's sticky note and node name say 'every 5 minutes', but the export stores only the unit ('minutes'), not the number. This cannot be confirmed from the uploaded workflow configuration.
- **AI:** only CT-01 uses AI: one AI Agent node, a Google Gemini chat model (temperature 0.2), Postgres chat memory and a structured output parser. The JSON shows **no tools** connected to the agent.
- **External services seen in the JSON:** Supabase (database), Slack, Gmail, HubSpot, Google Gemini, a Postgres database for AI memory.
- **Nothing was executed.** This is an explanation of the configuration, not a test result.

2. Four findings worth knowing straight away

These are confirmed by the JSON itself. They are explained fully later (Part 7), but they change how you should read everything else.

Audit flag
1. CT-04 (error handler) is only attached to CT-06. CT-04's note says it should be the Error Workflow on CT-01, CT-05 and CT-06. Only CT-06's settings contain the link (`errorWorkflow: maC3TWvbbZsLZAuZ`). CT-01 and CT-05 have none, so if either crashes, CT-04 will not run.

Audit flag
2. The HubSpot hook is switched off. In CT-02, the node that starts CT-07 has `"disabled": true`. As exported, CT-07 never runs. Final states still complete normally because a disabled node simply passes data through.

Audit flag
3. The AI's instructions and its output format disagree. The agent's system prompt asks for a 13-field result including a one-word category such as 'Logistics'. The attached output parser demands a different 6-field format with a fixed category list. The code downstream reads only the parser's fields.

Audit flag
4. Neither webhook has authentication configured. CT-01's sticky note says 'Authenticate', but the webhook node has no authentication option set. CT-06 is the same. Anyone who knows the URL could post to them. Whether a proxy or firewall in front protects them cannot be confirmed from the JSON.

3. The real-life analogy: a company operations control room

Imagine a control room that receives reports about business problems, such as a delayed shipment or a payment issue. Each person in the room matches one part of your system.

Control room role	n8n workflow	What they do
Front desk and registrar	CT-01 (first half)	Checks the report is complete, stamps it with a fingerprint, refuses duplicates, opens a case file
Analyst	CT-01 (AI Agent)	Writes a short impact assessment of the problem
Rules clerk	CT-01 (Code node)	Uses a fixed formula, not the analyst, to score urgency and decide whether a manager must sign off
Manager sign-off	CT-01 (Gmail approval)	Emails an approver who has 45 minutes to click Approve or Reject
File clerk	CT-02	Every change to a case file goes through this clerk, who validates it and records it
Messenger	CT-03	Sends the alert to the Slack channel
Night watchman	CT-05	Every few minutes: retries failed deliveries, flags stuck cases, notices 'three fires in one department'
Emergency phone	CT-04	Rings Slack when a workflow itself breaks
Reopen desk	CT-06	A human says 'restart case number 42'
CRM note-taker	CT-07	Would add a note to the customer's HubSpot record (currently switched off)

4. What the whole system does

1. What enters. A JSON message (structured text made of label: value pairs) from another system. It contains `event_id`, `timestamp`, `department`, `event_type`, `severity`, `entity_id` and a `payload` with extra details. A `payload.customer_email` is optional.

2. What the system does with it. It validates the message, creates an **incident** (a case file) in the database, has the AI assess it, calculates a priority and risk score, asks a human for approval if needed, and posts an alert to Slack.

3. Decisions it makes. Duplicate or not; valid or not; approval needed or not; approved, rejected or timed out; Slack succeeded or failed; retries exhausted or not.

4. Where information is stored. A Supabase database holds the tables `incidents`, `audit_logs`, `approvals` and `system_config`. A Postgres database holds the AI's chat memory. HubSpot would hold notes if CT-07 were switched on.

5. Who receives information. Slack (alerts), Gmail (approval requests and escalation emails), HubSpot (outcome notes, currently off), and the caller of the webhook (instant 200, 202 or 400 replies).

6. The result. Every incident ends as COMPLETED or REJECTED, or as ESCALATED or FAILED if a human has to step in. Every change is recorded in `audit_logs`.

The incident life-cycle

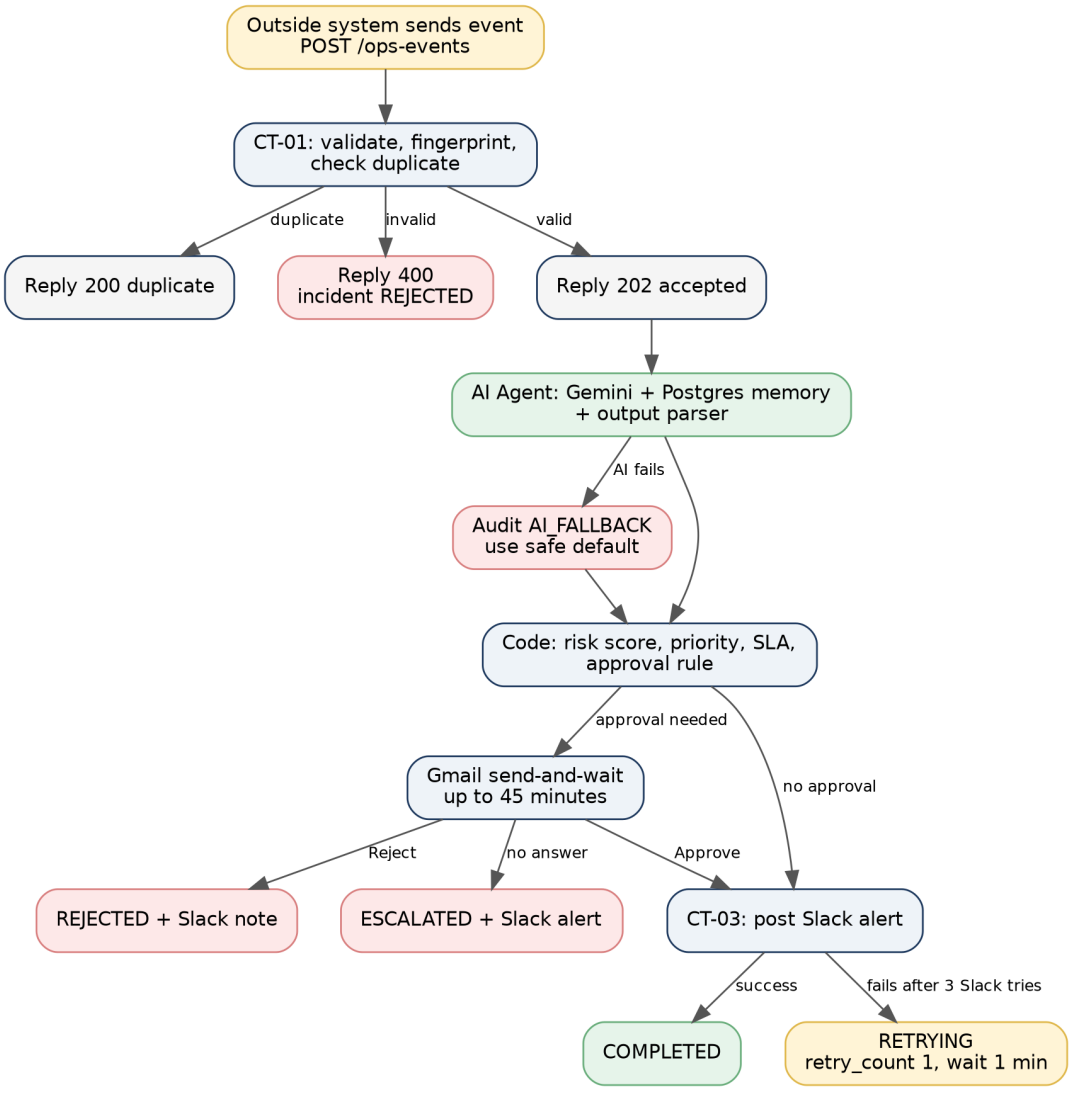
The normal route is:

```
RECEIVED -> VALIDATING -> AI_PROCESSING -> CLASSIFIED -> (AWAITING_APPROVAL -> APPROVED) -> PROCESSING -> COMPLETED
```

The side exits are REJECTED, RETRYING, ESCALATED and FAILED. CT-02 enforces which moves are legal. 'State' simply means 'which stage this case file is currently at'.

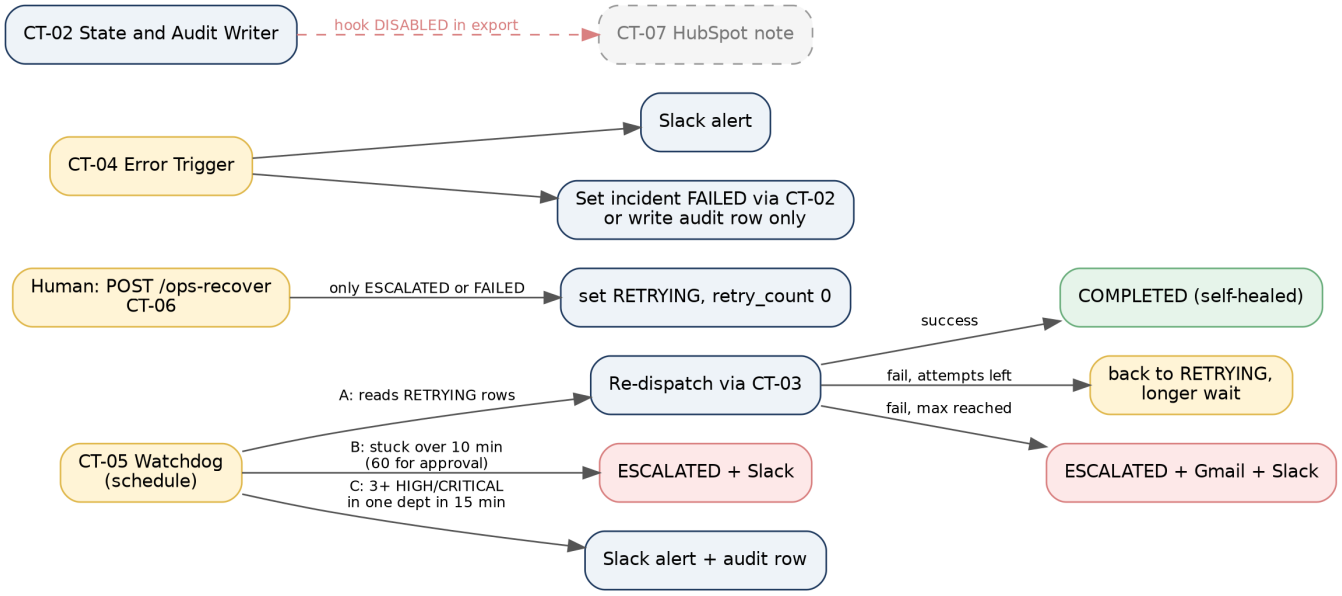
5. Overall diagrams

Diagram 1: the main journey. Boxes show real step names from CT-01 and its helpers. Every arrow exists in the JSON.



Reading it: a green box is a good ending, a red box is a bad or stopped ending, a yellow box is a waiting or retry state.

Diagram 2: the support workflows. The dashed arrow is the HubSpot hook that is switched off in the export.



Good to know
CT-05 and CT-06 do not call CT-01. They work by changing or reading rows in the `incidents` table, and CT-02/CT-03 do the real work. This is the 'shared database' pattern explained in section 6.

6. How the workflows talk to each other

There are three different mechanisms in your files. It matters which is which.

6.1 Direct calls (Execute Workflow nodes)

One workflow phones another and, by default, waits for the answer.

Caller	Called	How many call nodes	Waits for answer?	Notes
CT-01	CT-02	11	Yes	No error output on these calls; a CT-02 failure fails CT-01
CT-01	CT-03	1	Yes	Error output on: a failure leads to the retry path
CT-05	CT-02	4	Yes	Sets PROCESSING, COMPLETED, retry result and ESCALATED
CT-05	CT-03	1	Yes	Error output on: a failure leads to retry or escalation
CT-04	CT-02	1	Yes	Sets FAILED when a crash is linked to an open incident
CT-06	CT-02	1	Yes	Sets RETRYING after a human recovery request
CT-02	CT-07	1	**No** (fire and forget)	**Node is disabled** in the export, so no call is made

6.2 Through the database (no direct call)

CT-01 sets an incident to RETRYING with a `next_retry_at` time. Later, CT-05 reads the `incidents` table, finds those rows and retries them. CT-06 does the same: it sets RETRYING and CT-05 picks it up. Neither workflow calls

the other. The table is the link, like a task board that two people check.

6.3 Through n8n's own settings

CT-06's settings point to CT-04 as its Error Workflow. If CT-06 crashes, n8n itself starts CT-04. This is not a node-to-node call. CT-01 and CT-05 have no such setting in their exports (see finding 1).

6.4 The two webhooks

CT-01 (`/ops-events`) and CT-06 (`/ops-recover`) are entry points for **outside** systems or people over the internet. A **webhook** is an electronic reception point: another system sends information to a web address and the workflow starts. The JSON does not show which outside systems send to them. That cannot be confirmed from the uploaded workflow configuration.

7. Roadmap: the full guide

The full explanation is too large for one document, so it is split into seven numbered parts. No node or connection is skipped.

Part	Contents
1 (this file)	Inspection, four early findings, analogy, system summary, diagrams, how workflows communicate
2	n8n basics (items, JSON, expressions), then every node in CT-02 and CT-03
3	Every node in CT-01: intake and AI stage
4	Every node in CT-01: scoring, approval, dispatch and retry stage
5	Every node in CT-05, CT-06, CT-04 and CT-07
6	Data movement, workflow communication in detail, decisions, AI/database/CRM, error handling, simulated end-to-end examples
7	Data flow tables, glossary, full audit with fixes, test plan, one-page teaching summary

Plain English
Each part reuses the same node layout: what it is, why it is needed, an everyday comparison, what enters, what happens inside, what leaves, where it goes next, what happens on failure, and why it matters.